

Rapport de projet

Titre Professionnel Concepteur Développeur d'Applications

Lifequest - Application de gestion financière personnelle

Candidat : Antonin MINGAM

Titre visé : Concepteur Développeur d'Applications Niveau 6

Référentiel : TP-01281 millésime 04 (24/05/2023)

Sommaire

1. [Présentation du candidat et du contexte de formation](#)
2. [Présentation du projet LifeQuest](#)
3. 2.1 [Contexte et objectifs](#)
4. 2.2 [Stack technique](#)
5. 2.3 [Architecture générale](#)

BLOC 1 — Développer une application sécurisée

1. [CP1 — Installer et configurer son environnement de travail en fonction du projet](#)
2. [CP2 — Développer des interfaces utilisateur](#)
 3. 4.1 [Conception des interfaces avec Phoenix LiveView](#)
 4. 4.2 [Charte graphique et responsive design \(Tailwind CSS / daisyUI\)](#)
 5. 4.3 [Accessibilité \(RGAA\)](#)
 6. 4.4 [Conformité RGPD — Mentions légales](#)
 7. 4.5 [Tests des composants d'interface](#)
 8. 4.6 [Sécurité des interfaces \(XSS, CSRF\)](#)
9. [CP3 — Développer des composants métier](#)
 10. 5.1 [Architecture en contextes \(Finances, Accounts\)](#)
 11. 5.2 [Développement défensif et gestion des erreurs](#)
 12. 5.3 [Communication en temps réel avec Phoenix PubSub](#)
 13. 5.4 [Tests unitaires des composants métier](#)
 14. 5.5 [Sécurité des composants métier](#)
15. [CP4 — Contribuer à la gestion d'un projet informatique](#)
 16. 6.1 [Méthode de développement itérative](#)
 17. 6.2 [Planification et suivi des tâches \(Jira\)](#)
 18. 6.3 [Outils collaboratifs \(GitHub, conventions de commits\)](#)
 19. 6.4 [Qualité du code \(Credo, mix format, CI\)](#)

BLOC 2 — Concevoir et développer une application sécurisée organisée en couches

1. [CP5 — Analyser les besoins et maquetter une application](#)
2. 7.1 [Analyse du cahier des charges et identification des besoins](#)
3. 7.2 [User stories](#)
4. 7.3 [Maquettes et enchaînement des écrans](#)
5. [CP6 — Définir l'architecture logicielle d'une application](#)
6. 8.1 [Architecture multicouche \(web / contextes / schémas\)](#)
7. 8.2 [Choix du framework et de l'ORM \(Phoenix / Ecto\)](#)
8. 8.3 [Stratégie de sécurité par couche](#)
9. 8.4 [Éco-conception](#)
10. [CP7 — Concevoir et mettre en place une base de données relationnelle](#)
11. 9.1 [Modèle Conceptuel de Données \(MCD\)](#)
12. 9.2 [Modèle Logique de Données \(MLD\)](#)
13. 9.3 [Migrations Ecto et scripts de création](#)
14. 9.4 [Sécurité et droits d'accès](#)
15. 9.5 [Jeu d'essai](#)
16. [CP8 — Développer des composants d'accès aux données SQL et NoSQL](#)
 - 10.1 [Requêtes Ecto \(CRUD sécurisé\)](#)
 - 10.2 [Gestion de l'intégrité et des conflits d'accès \(transactions\)](#)
 - 10.3 [Validation et contrôle des entrées côté serveur](#)
 - 10.4 [Tests unitaires et de sécurité des accès aux données](#)
 - 10.5 [Protection contre l'injection SQL](#)

BLOC 3 — Préparer le déploiement d'une application sécurisée

1. [CP9 — Préparer et exécuter les plans de tests d'une application](#)
 - 11.1 [Plan de tests](#)
 - 11.2 [Tests unitaires \(ExUnit\)](#)

- 11.3 [Tests d'intégration \(LiveView\)](#)
 - 11.4 [Tests de sécurité](#)
 - 11.5 [Résultats et compte-rendu de tests](#)
2. [CP10 — Préparer et documenter le déploiement d'une application](#)
- 12.1 [Environnements \(développement, test, production\)](#)
 - 12.2 [Procédure de déploiement](#)
 - 12.3 [Scripts de déploiement et migrations](#)
 - 12.4 [Variables d'environnement et configuration](#)
3. [CP11 — Contribuer à la mise en production dans une démarche DevOps](#)
- 13.1 [Pipeline CI/CD \(GitHub Actions\)](#)
 - 13.2 [Conteneurisation \(Docker\)](#)
 - 13.3 [Outils de qualité de code \(Credo, mix format\)](#)
 - 13.4 [Automatisation des tests](#)
 - 13.5 [Interprétation des rapports CI](#)

Compétences transversales

1. [Communiquer en français et en anglais](#)
- 14.1 [Documentation technique en anglais](#)
 - 14.2 [Internationalisation avec Gettext](#)
 - 14.3 [Lecture de documentation technique en anglais](#)
2. [Mettre en œuvre une démarche de résolution de problème](#)
- 15.1 [Refactoring du modèle de données \(LQ-5\)](#)
 - 15.2 [Encodage des apostrophes dans les tests LiveView](#)
 - 15.3 [Diagnostic d'une CI rouge inattendue](#)
3. [Apprendre en continu — Veille technologique](#)
- 16.1 [Sources suivies](#)
 - 16.2 [Format et méthode](#)
 - 16.3 [Exemple concret : adoption de Tailwind CSS v4](#)

4. [Conclusion et bilan de compétences](#)

- 17.1 [Bilan technique](#)
- 17.2 [Bilan personnel](#)
- 17.3 [Suite et perspectives](#)

1. Présentation du candidat et du contexte de formation

Avant d'entrer en formation, mon parcours étudiant et professionnel a été un long processus exploratoire. Après une scolarité exemplaire, j'ai traversé plusieurs filières classiques qui ne me correspondaient pas, avant de faire le choix d'une carrière mêlant technique et artistique, en tant qu'ingénieur du son. J'ai exercé cette activité pendant 5 ans en autoentreprise, acquérant des compétences techniques et transversales en autonomie et en gestion d'entreprise.

Mon choix de changer de direction arrive à la rencontre de plusieurs événements simultanés dans ma vie d'entrepreneur :

- la sensation d'avoir atteint un plafond de verre
- la frustration de ne pas avoir de diplôme qualifiant
- l'arrivée imminente de mes 30 ans
- la rencontre avec le monde du développement

En effet, malgré une rapide découverte au cours de mon adolescence, je n'avais jamais vraiment compris ce monde si vaste et riche. Lors de mon parcours entrepreneurial, j'ai été amené à rédiger des newsletters en HTML/CSS d'abord, puis à vouloir refondre mon site web professionnel, initialement créé avec un outil no-code. J'ai fini par consacrer une grande partie de mon temps libre à l'apprentissage des bases du développement, grâce aux cours en ligne d'OpenClassrooms puis de l'université d'Harvard. Cette montée en compétences progressive m'a convaincu de mon attrait pour le développement, et de ma motivation à concrétiser ces nouvelles compétences grâce à des études traditionnelles.

C'est dans la suite logique de cette formation autodidacte que j'ai décidé d'intégrer la 2iAcademy en Conception et Développement d'Applications, en alternance chez

Frixel, afin de renforcer mes compétences et d'en acquérir de nouvelles en conception, ainsi que d'obtenir un diplôme reconnu et qualifiant.

À l'issue de cette formation, je poursuivrai au sein de l'ESGI Paris en Mastère Big Data & Intelligence Artificielle dans le but de me spécialiser dans un secteur en forte croissance. À plus long terme, je me réserve le droit de retrouver le chemin de l'entrepreneuriat, fort d'une double compétence technique et d'un diplôme reconnu.

2. Présentation du projet LifeQuest

2.1 Contexte et objectifs

Durant mon activité entrepreneuriale, il était très compliqué de me projeter dans l'avenir à moyen ou long terme. Mon arrivée dans un univers professionnel stable m'a permis de découvrir la gestion d'épargne à long terme, les objectifs financiers et les placements financiers. En explorant le sujet, j'ai découvert beaucoup d'applications de gestion financière et de placements financiers, mais aucune qui permettait de faire les deux à la fois, en plus d'offrir du conseil en gestion de patrimoine. Des applications comme Bankin' ou Linxo se concentrent sur le suivi bancaire et la catégorisation des dépenses, tandis que Finary ou Trade Republic se positionnent sur la gestion de placements. Aucune n'articule les deux en les ancrant autour d'objectifs de vie personnels, avec un moteur de simulation intégré. Un ami travaillant dans la finance m'avait fait part de son idée d'application de gestion et de conseil financiers, et une connaissance japonaise m'a fait découvrir des outils de gestion financière répandus au Japon. Ces différents échanges ont fait émerger l'idée de LifeQuest, avec une approche innovante articulée autour des objectifs de vie de l'utilisateur, comme un achat de maison ou la volonté d'avoir un enfant. L'utilisateur renseigne sa situation financière actuelle, une estimation de ses revenus et de ses dépenses mensuels. Cela nous permet de prendre une photo de son patrimoine et d'effectuer des projections du futur simplement. Il définit ensuite un objectif représenté par un montant cible et une échéance. À partir de ces données, l'application génère des recommandations personnalisées : réduction des dépenses non essentielles, placements ponctuels ou réguliers, avec un niveau de risque adapté à la complexité de l'objectif et à la situation financière déclarée.

L'application cible des particuliers souhaitant gérer leur budget personnel et planifier des objectifs financiers à moyen ou long terme, sans nécessiter de connaissances préalables en finance.

2.2 Stack technique

- Backend : Elixir 1.18 / OTP 27, Phoenix 1.8, Phoenix LiveView 1.1, Ecto
- Base de données : PostgreSQL 16
- Frontend : Tailwind CSS v4, daisyUI, templates HEx (pas de framework JS séparé)
- Tests : ExUnit, ConnCase, Phoenix.LiveViewTest
- CI : GitHub Actions

Elixir est un langage fonctionnel, concurrent et tolérant aux pannes, qui s'exécute sur la BEAM, la machine virtuelle d'Erlang éprouvée depuis des décennies dans les télécoms. Phoenix est le framework web Elixir de référence, qui embarque LiveView pour la construction d'interfaces réactives côté serveur.

Le choix de cette stack technique s'est présenté comme une évidence, car c'est celle que j'ai apprise et sur laquelle j'ai travaillé durant la majorité de mon alternance chez Frixel. Le framework Phoenix est très complet, innovant, et bien conçu. La séparation des responsabilités est très claire (contexts, schemas, controllers, LiveViews), les outils intégrés sont très riches, et LiveView permet de construire des interfaces réactives côté serveur sans avoir à maintenir un framework JavaScript séparé.

Côté données, Phoenix fournit l'ORM Ecto, conçu pour fonctionner par défaut avec PostgreSQL, avec un système de migrations versionné et les changesets qui permettent la validation des données avant toute écriture en base.

`mix phx.gen.auth` nous fournit un système d'authentification poussé en une commande, avec stockage des mots de passe hashés avec bcrypt en base, ainsi que le magic link, un système de vérification de compte et de réinitialisation de mot de passe grâce à un envoi de token par email.

Les tests s'appuient sur ExUnit pour les tests unitaires, ConnCase pour les routes HTTP et Phoenix.LiveViewTest pour les interactions LiveView.

L'intégration continue est assurée par GitHub Actions, dont le pipeline exécute les tâches d'installation et de validation fournies par l'écosystème Phoenix : `mix format --check-formatted` et `mix credo --strict` pour le linting et le formatage, `mix test` pour l'exécution des tests et `mix compile --warnings-as-errors` pour la compilation stricte du projet.

2.3 Architecture générale

LifeQuest repose sur une architecture en trois couches strictement séparées, afin de respecter la convention proposée par le framework.

Couche présentation : HEx est un langage de templating permettant de construire nos pages webs en Elixir. Ce sont les LiveViews qui sont chargées d'interpréter ce code. Lors de leur création, un état est généré côté serveur et une Websocket persistante est ouverte pour communiquer avec le navigateur. Les interactions utilisateur déclenchent des événements traités par le serveur qui renvoie le diff partiel du DOM, permettant de changer l'interface sans rechargement de la page.

Couche métier : Ce sont les contextes Elixir qui portent ces responsabilités. Chaque contexte expose des fonctions publiques qui pourront être appelées par l'ensemble des LiveView. Ces dernières doivent passer par les fonctions des contextes pour interagir avec la base de données. Cela centralise la logique métier, et facilite les tests.

Couche données : Les schémas Ecto définissent la structure des données et les changesets qui valident les entrées avant toute écriture. Ecto génère des requêtes paramétrées afin d'éviter les injections SQL.

Une autre fonctionnalité offerte par Phoenix s'appelle Scope. Il s'agit d'une struct représentant l'utilisateur actuel qui est passée en premier argument à toutes les fonctions de contexte. Cela garantit l'isolation des données entre les utilisateurs, car elle est utilisée comme filtre dans toutes les requêtes Ecto. On peut également s'appuyer sur elle pour la gestion des droits, par exemple entre admin et user, ou entre compte free ou premium.

Pour résumer, à l'appel d'une route : **Voir Annexe 1.1 — schéma UML de la séquence de traitement d'une requête**

- la requête HTTP initiale déclenche le `mount/3` de la LiveView
- les fonctions de contextes nécessaires au rendu initial sont appelées
- les requêtes SQL sont envoyées à PostgreSQL grâce à Ecto
- les données sont placées en `assigns` sur la socket et stockées en mémoire vive
- le template HEx s'affiche, en utilisant ces données

Par la suite, les interactions utilisateurs déclencheront des fonctions `handle_event/3` qui fonctionnent de manière similaire, et qui permettront de modifier le DOM sans rechargement ou d'émettre des messages PubSub pour avertir d'autres LiveViews abonnées de la mise à jour de leur état en temps réel grâce à `handle_info/3`

3. CP1 — Installer et configurer son environnement de travail en fonction du projet

IDE et extensions

Pour le développement, j'ai utilisé VSCode car c'est l'IDE que j'ai l'habitude d'utiliser. Son catalogue d'extensions m'a permis d'installer des aides au développement adaptées à la stack : ElixirLS pour l'autocomplétion et la navigation dans le code Elixir, l'extension PlantUML pour la création et la visualisation de mes schémas UML directement dans l'éditeur, et TODO Highlight pour garder en tête les tâches à effectuer dans le code.

Outillage Mix

Le projet repose principalement sur Elixir et Mix, son gestionnaire de tâches natif. J'ai créé une tâche personnalisée, `mix precommit`, afin de grouper les commandes de formatage, de lint, de build et d'exécution des tests et de m'assurer de la qualité de mon code avant de le pousser sur la branche distante. Toutes ces étapes sont identiques à celles exécutées sur la pipeline CI à la création d'une pull request, ce qui me permet de capturer les problèmes plus tôt dans mon workflow.

`mix setup` permet d'installer le projet et ses dépendances, créer les bases de données et jouer leurs migrations. On dispose de deux bases de données par défaut : la base de développement et la base de tests, permettant d'effectuer des tests reproductibles sur cette dernière sans affecter les données de développement.

Commandes de lancement

`mix phx.server` permet de lancer l'application en mode développement sur le port 4000 de la machine hôte. `iex -S mix phx.server` lance en parallèle un shell interactif, utile pour tester des expressions Elixir en contexte d'application.

Même si j'ai travaillé seul sur ce projet, j'ai également utilisé les outils de versioning Jira, Git, Github, de CI GitHub Actions de conteneurisation Docker et d'orchestration. Mon utilisation de ces outils seront détaillés dans les parties de ce rapport qui leurs sont consacrées.

4. CP2 — Développer des interfaces utilisateur

4.1 Conception des interfaces avec Phoenix LiveView

LifeQuest repose entièrement sur Phoenix LiveView pour la construction de ses interfaces. Ce modèle se distingue des approches frontend classiques (React, Vue) en maintenant l'état de l'interface côté serveur : le navigateur établit une connexion WebSocket persistante avec le serveur, qui calcule les changements de DOM et envoie uniquement les diffs nécessaires au client. Cette architecture élimine le besoin d'un framework JavaScript séparé et d'une API entre le front et le back, tout en offrant une réactivité comparable à celle d'une Single Page Application.

Chaque LiveView suit une structure définie par trois callbacks principaux :

- `mount/3` initialise l'état de la page en chargeant les données nécessaires dans les assigns.
- `handle_event/3` traite les interactions utilisateur comme les clics ou les soumissions de formulaire et met à jour l'état en conséquence.

- `handle_info/3` reçoit les messages asynchrones, notamment ceux émis via PubSub.

Le template HEx associé reflète automatiquement chaque changement d'état sans rechargement de page.

Il est possible de factoriser les composants réutilisables dans des modules personnalisés, pour permettre une organisation propre à chaque équipe.

L'implémentation de base de Phoenix les regroupe dans `core_components.ex`, qui expose des composants fonctionnels tels que `<.input>`, `<.form>` et `<.icon>`. Cette centralisation garantit la cohérence visuelle et comportementale à travers l'application : un champ de formulaire se comporte et s'affiche de manière identique partout, et toute modification s'applique en un seul point.

Exemple concret de la LiveView `dashboard_live` : Voir annexe 2.1

A l'initialisation de la LiveView, la fonction `mount/3` est appelée, et appelle elle-même les différentes fonctions du contexte permettant de récupérer les données en base permettant l'affichage des données sur le dashboard, avant de les placer dans les assigns. Parmi elles, la fonction `load_dashboard_data` qui vient appeler différentes fonctions du contexte `Finances` pour récupérer les revenus et les dépenses de l'utilisateur en base. Cela permet de générer les deux graphiques. Si des revenus ou des dépenses récurrentes ont été générées automatiquement en base sans avoir été validées, elles apparaissent dans une section de la page, `pending_recurring_section`, permettant à l'utilisateur de les valider une par une. En cliquant sur le bouton valider, l'évènement `validate_recurring` est déclenché et capté par un `handle_event` qui va lui-même essayer d'appeler la fonction `validate_recurring/3` du contexte `Finances` pour mettre à jour les informations dans les assigns de la socket. La transaction perd son état pending, elle disparaît donc de la section dédiée à ces dernières.

4.2 Charte graphique et responsive design (Tailwind CSS / daisyUI)

Pour la couche de présentation, j'ai utilisé Tailwind CSS v4 associé à daisyUI, qui sont les outils fournis par Phoenix et que j'utilise au quotidien chez Frixel.

Tailwind est un framework CSS utility-first : plutôt que de définir des classes sémantiques, on compose les styles directement dans le HTML grâce à des classes atomiques (`flex` , `gap-4` , `rounded-lg` ...). Cela évite d'écrire du CSS custom et élimine les conflits de nommage.

A la compilation, seules les classes effectivement utilisées sont incluses dans le bundle final, ce qui allège le poids des assets en production.

daisyUI vient compléter Tailwind en proposant des composants prêts à l'emploi : `btn` , `card` , `badge` , `menu` ... Ce sont de simples classes CSS qui regroupent des combinaisons de classes Tailwind, sans JavaScript embarqué.

Le système de thèmes est configuré dans `app.css` où l'on définit nos couleurs `primary` , `base-100` , `base-200` ... Changer le thème revient à changer la valeur de ces variables, sans toucher aux composants. LifeQuest propose trois modes : clair, sombre, et système. Ce dernier détecte automatiquement la préférence du navigateur.

La bascule est gérée côté client via l'attribut `data-theme` sur la balise `html` , ce qui évite un rechargement de page.

Pour le responsive, j'ai utilisé les préfixes de breakpoints de Tailwind (`sm:` , `md:` , `lg:`), qui permettent de modifier les valeurs voulues selon la taille de l'écran inline, sans avoir à écrire de media-queries.

4.3 Accessibilité (RGAA)

Pour travailler sur l'accessibilité, je me suis basé sur les retours d'audit de Lighthouse sur chacune des pages de l'application, ce qui m'a amené à étudier les attributs aria manquants et les contrastes de couleur.

Les composants daisyUI fournissent des labels aria de base sur les éléments courants. J'ai ajouté les labels manquants sur les boutons sans texte.

De plus, sur certains éléments qui se répètent sur une même page, j'ai ajouté un label dynamique. Par exemple sur les icônes de modification et de suppression des différentes transactions de la page "Finances" j'ai inclu le libellé de la transaction ciblée, ce qui donne des labels comme "Modifier Loyer" ou "Supprimer Salaire net", permettant à l'utilisateur de distinguer chaque bouton sans ambiguïté.

Toutes les pages de l'application ont un score d'accessibilité supérieur à 90 suite aux audits Lighthouse.

Certaines interactions complexes comme les graphiques n'ont pas fait l'objet de tests spécifiques avec des lecteurs d'écran.

4.4 Conformité RGPD — Mentions légales

LifeQuest collecte et traite des données personnelles, ce qui implique de respecter le Règlement Général sur la Protection des Données. J'ai créé une page dédiée, accessible depuis le footer sur l'ensemble de l'application, y compris sans être connecté.

La base légale retenue est l'exécution d'un contrat au sens de l'article 6(1)(b) du RGPD : en créant un compte, l'utilisateur accepte que ses données soient traitées pour lui permettre d'utiliser le service. Les données sont conservées pendant toute la durée d'activité du compte, puis supprimées à sa résiliation. Les tokens d'authentification expirent automatiquement après 60 jours.

La page liste les droits de l'utilisateur : accès, rectification, effacement, portabilité et opposition. Le droit à l'effacement est directement accessible depuis les paramètres du compte. La suppression du compte entraîne la suppression en cascade de toutes les données associées en base.

4.5 Tests des composants d'interface

Les interfaces de LifeQuest sont testées via le module `Phoenix.LiveViewTest` qui simule le cycle de vie complet d'une LiveView dans un environnement de test.

Les tests utilisent le module `ConnCase` pour établir une connexion HTTP simulée avec une session authentifiée.

La stratégie de test porte sur trois aspects : la vérification du rendu HTML attendu, la présence des éléments d'interface critiques et le comportement en réponse aux interactions utilisateur.

`describe` permet de grouper des tests unitaires qui partagent la même situation initiale `test` est un test unitaire, qui contient nos assertions `assert html =~ "texte attendu"` permet de vérifier la présence d'un contenu textuel dans le

rendu `assert has_element?(live_view, "sélecteur CSS")` permet de vérifier la présence d'un élément spécifique dans le DOM.

Exemple concret des tests de la LiveView `dashboard_live` : Voir annexe 2.1

`setup :register_and_log_in_user` permet de simuler la situation initiale : un utilisateur est connecté. C'est nécessaire car toutes les opérations testées ne sont déclenchables que dans cette situation précise.

`create_recurring_last_month(scope, attrs \\ %{})` est une fonction qui permettra de créer une opération récurrente au besoin. Des valeurs par défaut sont définies, mais peuvent être écrasées en fournissant le deuxième argument facultatif.

`test "shows pending recurring incomes from last month"` crée une opération récurrente à valider par défaut et s'assure qu'elle s'affiche bien sur la page

`test "does not show recurring already validated this month"` crée une opération récurrente et insère sa validation en base pour le mois courant, puis vérifie qu'elle n'apparaît bien pas dans la liste des revenus à valider

`test "validate_recurring duplicates transaction to current month"` simule la validation manuelle par l'utilisateur, et s'assure que cela entraîne la disparition de la ligne sur la page

4.6 Sécurité des interfaces (XSS, CSRF)

Phoenix fournit des protections contre les principales vulnérabilités web directement dans sa couche de templating. Contre le Cross-Site Scripting (XSS), HEEx échappe automatiquement toute variable interpolée via `{@assign}` : si une donnée utilisateur contient du HTML ou du JavaScript, le contenu est affiché tel quel, sans être interprété par le navigateur.

Aucune balise `<script>` inline n'est présente dans les templates HEEx de LifeQuest : l'ensemble du JavaScript est isolé dans le répertoire `assets/js/`.

Contre les attaques Cross-Site Request Forgery (CSRF), Phoenix génère un token unique intégré à chaque formulaire via le composant `<.form>`. Ce token est

vérifié côté serveur à chaque soumission, ce qui garantit que la requête provient bien de l'application et non d'un site tiers.

Les Scopes et leur présence dans les arguments des fonctions de Repo garantissent que l'utilisateur actif peut récupérer les données lui appartenant, et seulement celles-ci.

5. CP3 — Développer des composants métier

5.1 Architecture en contextes (Finances, Accounts)

Les contextes sont un pilier d'organisation du code de Phoenix. Ils regroupent la logique métier dans des modules, qui exposent une API publique que la couche présentation peut utiliser. Dans mon application, deux contextes ont été mis en place, `Accounts` qui gère l'authentification et le Scope, et `Finances` qui contient la logique métier de l'application.

Ce dernier expose notamment les fonctions CRUD générées à la création du schéma ainsi que toutes les fonctions d'accès aux données plus spécifiques. C'est le seul moyen pour les LiveViews de communiquer avec l'ORM, elle n'appellent jamais `Repo` directement, et j'ai appliqué cette contrainte strictement dans l'ensemble du projet.

Les `Scopes`, apportés par Phoenix 1.8 et générés par `phx.gen.auth` permettent d'isoler les données de manière sécurisée. L'utilisateur actif est passé en premier argument de toutes les fonctions de contexte, et est utilisé pour filtrer les requêtes Ecto. De cette façon, on est certain qu'un utilisateur ne peut jamais accéder aux données d'un autre.

5.2 Développement défensif et gestion des erreurs

Pour présenter cette partie, laissez moi vous parler du concept de pattern matching. Il s'agit d'une fonctionnalité Elixir qui permet à des fonctions qui portent le même nom d'avoir des corps différents selon leur arité ou selon la forme des arguments reçus. Quand une fonction avec plusieurs définitions est appelée, Elixir essaye de pattern match avec chacune d'entre elle dans l'ordre du fichier et exécute la première qui correspond. Il est très commun de terminer la liste

d'implémentation d'une fonction par une "catch all", qui permet d'avoir un comportement par défaut si aucune des implémentations spécifiques ne correspond.

Ecto s'appuie sur cette mécanique pour générer ses valeurs de retour grâce à des objets appelés `changesets`. Ils sont utilisés pour s'assurer de la validité des données côté serveur. Si une validation échoue, l'écriture en base est annulée et un tuple `{:error, _changeset}` est retourné, et le `changeset` contient les messages d'erreur, permettant de les afficher directement sur le formulaire sans recharger la page ni crash.

5.2.a Exemples concrets de pattern matching : Voir annexe 3.1

La fonction `format_transaction_type/1` du Dashboard a deux clauses, l'une qui matche sur `%{direction: :income}` et l'autre sur `%{direction: :expense}`. Dans les deux cas, l'objet passé en argument doit être un objet qui contient ces deux types, en l'occurrence une transaction. Appeler cette fonction avec une transaction revenu ou une transaction dépense déclenche automatiquement la bonne clause, sans `if` ni `cond`.

La fonction `format_income_type/1` prend le relais pour les revenus, et pattern match sur le type de transaction pour renvoyer une chaîne de caractère à afficher dans le DOM. La clause catch all, représentée par le symbole `_` gère les cas qu'on aurait pas prévu sans faire crasher l'application.

Ce principe de pattern matching permet aussi d'adapter le comportement de la fonction dépendamment de la valeur de retour d'un appel de fonction présent dans son corps. Cet `handle_event/3` lance un case pour appeler `Finances.validate_recurring/3`, et pattern match sur `{:ok, }` et `{:error, }` pour effectuer la suite de ses opérations. Les deux cas sont traités explicitement, et aucun chemin d'exécution n'est ignoré silencieusement. C'est un pattern extrêmement commun en Elixir.

5.3 Communication en temps réel avec Phoenix PubSub

Phoenix PubSub permet à des processus de s'abonner à des canaux nommés et de recevoir des messages asynchrones émis par d'autres processus. Dans LifeQuest, je l'utilise pour propager les mises à jour de données entre LiveViews.

L'abonnement se fait dans `mount/3` au moment de la connexion à la socket. Les canaux sont scopés par utilisateur, avec un nom du type `"user:#{user_id}:transactions"`. Un message publié sur ce canal n'est reçu que par les LiveViews de cet utilisateur précis. Cela garantit que les événements d'un utilisateur ne se propagent pas aux sessions d'un autre.

Quand un message PubSub est reçu, c'est le callback `handle_info/2` qui le traite. Il fonctionne de la même manière qu'un `handle_event/3`, mais déclenché par un message interne plutôt que par une action utilisateur.

5.4 Tests unitaires des composants métier

La séparation entre contexte et LiveView a un avantage direct pour les tests : on peut tester entièrement la logique métier de `Finances` sans instancier de LiveView ni simuler de connexion HTTP.

Les contextes `Finances` et `Accounts` sont testés dans `test/lifequest/finances_test.exs` et `test/lifequest/accounts_test.exs` en suivant la convention Phoenix. Chaque fichier est structuré de la même façon : chaque fonction est décrite par un bloc `describe` qui contient le cas nominal, les cas d'erreur, et le test d'isolation par `Scope`.

Ce dernier est systématique. Pour chaque fonction qui récupère des données en base, on crée un second utilisateur et s'assure que l'appel de fonction ne retourne pas de données appartenant au premier. Ce test permet de garantir que les données ne peuvent pas fuiter entre les utilisateurs. **Voir annexe 3.2**

Pour créer facilement des données de tests sans dupliquer de code, Phoenix préconise de créer des fonctions helpers appelées fixtures. Ce sont des fonctions qu'on appelle dans chacun de nos tests pour définir des conditions initiales déterminées, avec des valeurs par défaut que l'on peut écraser selon les besoins du test. Cette approche rend les tests très lisibles, s'assure de leur reproductibilité et permet de se concentrer sur ce qu'ils vérifient.

5.5 Sécurité des composants métier

La sécurité des composants métier repose sur plusieurs couches complémentaires.

Les changesets Ecto sont construits à partir de formulaires, dont les champs sont transmis à une fonction `cast/3`. Cependant, les champs système comme le `user_id` ne passent jamais par cette fonction, et sont ajoutés automatiquement après la validation à partir du Scope contenu dans la socket. Cela empêche tout utilisateur malveillant de les manipuler pour accéder à des données qui ne leur appartiennent pas.

Les changesets incluent aussi des validations côté serveur systématiques : `validate_required` pour les champs obligatoires, `validate_number` pour les montants... On sait que la vérification côté client est utile pour l'expérience utilisateur mais n'est pas sécurisée, il est donc nécessaire de conserver cette étape. Il s'agit de notre source de vérité et de notre garantie que les données insérées en base sont toujours valides.

Les requêtes Ecto sont systématiquement filtrées grâce au Scope. On ne récupère jamais une donnée par son seul identifiant, on filtre toujours la requête par l'id de l'utilisateur grâce au pattern `where: t.account_id in ^account_ids`

On ne récupère jamais une transaction par son seul identifiant : la requête joint toujours l'identifiant du compte, soit en vérifiant que c'est bien la clé étrangère de la ligne, soit en faisant une jointure sur la table users si nécessaire.

Enfin, certaines données sont définies comme des Enum, ce qui permet de s'assurer que le contenu de la donnée appartient à une liste prédéfinie.

6. CP4 — Contribuer à la gestion d'un projet informatique

6.1 Méthode de développement itérative

Même si j'ai travaillé seul sur ce projet, j'ai appliqué les bonnes pratiques utilisées au quotidien chez Frixel afin de maintenir un historique propre, traçable et permettant le travail en équipe.

Chaque itération suit le même cycle : création du ticket Jira, ouverture d'une branche Git, développement, écriture des tests, création d'une Pull Request, validation par la CI, merge sur `main`.

L'application est restée fonctionnelle à chaque étape de développement, pratique importante si l'on souhaite la mettre à jour régulièrement sans avoir à arrêter l'application en production.

6.2 Planification et suivi des tâches (Jira)

Chaque fonctionnalité commence par un ticket Jira qui décrit le besoin et les critères d'acceptation, ce qui m'oblige à définir clairement ce que je veux produire avant de coder. Le numéro du ticket est ensuite porté par la branche Git, les commits et la Pull Request, ce qui rend l'historique entièrement traçable.

Le workflow Jira suit trois colonnes : backlog, en cours, terminé. Simple, mais suffisant pour garder une vision claire de l'avancement et prioriser les prochaines itérations.

6.3 Outils collaboratifs (GitHub, conventions de commits)

La suite du processus est la création d'une branche GIT selon la convention de nommage établie chez Frixel. Cette convention permet de relier directement le ticket, la branche et les merges sur main, eux même nommés en suivant une convention similaire.

`LQ-XX type(description)` :

- `LQ-XX` correspond au numéro de ticket JIRA
- `type` correspond à la nature de la modification (feat, fix, refactor...)
- `description` est un court résumé des changements apportés par la mise à jour

Une fois les modifications terminées, je push mon code sur une nouvelle branche distante sur GitHub puis je crée une Pull Request vers `main` ce qui déclenche automatiquement le pipeline CI grâce à GitHub Actions.

6.4 Qualité du code (Credo, mix format, CI)

Lorsqu'une PR est ouverte, une CI est automatiquement exécutée grâce à Github Actions.

Cette pipeline exécute trois vérifications primordiales : le formatage du code avec `mix format`, le lint avec `mix credo` qui applique des règles de lisibilité et de complexité définies en amont, l'exécution des tests avec `mix test`.

Ces trois vérifications sont enchaînées dans une tâche custom `mix precommit`, que j'exécute avant chaque commit, me permettant de détecter les problème avant de push mon code en distant.

Toutes ces étapes permettent de garantir de conserver un historique et une branche `main` propre, où chaque commit est validé en amont.

7. CP5 — Analyser les besoins et maquetter une application

7.1 Analyse du cahier des charges et identification des besoins

Comme expliqué en introduction, le point de départ de LifeQuest est un constat personnel : aucune application existante ne permet à la fois de suivre ses flux financiers mensuels et de simuler, planifier et concrétiser son avenir financier en fonction d'objectifs de vie concrets.

À partir de ce problème, j'ai identifié les besoins fonctionnels principaux :

- Saisie des revenus et dépenses par catégorie, avec support des transactions récurrentes
- Tableau de bord visuel synthétisant la situation du mois courant
- Projections financières à horizon personnalisable, basées sur les données déclarées
- Simulation de placements avec différents niveaux de risque
- Suivi d'objectifs financiers définis par l'utilisateur

Certaines fonctionnalités ont été volontairement mises hors périmètre pour garder le projet réaliste : le support multi-devises, la connexion à des comptes bancaires réels via API open banking et à des produits financiers réels pour faire de vrais placements.

7.2 User stories

Les user stories ont servi de fil directeur pour prioriser le développement. En voici quelques-unes représentatives :

En tant qu'utilisateur, je veux renseigner ma situation financière afin que l'application puisse calculer des projections réalistes. Critères : le formulaire accepte un montant d'épargne actuelle, de dettes et de revenu mensuel net. Les données sont sauvegardées et réutilisées dans toutes les projections.

En tant qu'utilisateur, je veux saisir mes revenus et dépenses par catégorie afin de connaître précisément la répartition de mes flux mensuels. Critères : je peux créer une transaction en choisissant un type (salaire, loyer, loisirs...), un montant et une date. Les transactions peuvent être marquées comme récurrentes.

En tant qu'utilisateur, je veux visualiser la répartition de mes dépenses sous forme de graphique afin d'identifier les postes les plus importants d'un coup d'oeil. Critères : le dashboard affiche deux donuts SVG, l'un pour les revenus et l'autre pour les dépenses, avec les montants par catégorie.

En tant qu'utilisateur, je veux simuler un placement mensuel sur plusieurs années afin de savoir combien j'aurai accumulé selon différents niveaux de rendement. Critères : je saisis un montant mensuel et un horizon en années. L'application affiche les projections pour trois scénarios : prudent, modéré et dynamique.

En tant qu'utilisateur, je veux supprimer mon compte et toutes mes données afin de pouvoir exercer mon droit à l'effacement. Critères : une action dans les paramètres supprime le compte et toutes les données associées en cascade, sans possibilité de récupération.

7.3 Maquettes et enchaînement des écrans

Les maquettes ont été réalisées en amont grâce à l'outil Figma. Elles sont inspirées des différentes applications que j'ai été amené à utiliser dans mon parcours, ou à étudier au début de mon étude de marché.

Elles couvrent les différents écrans de l'application, en suivant le parcours utilisateur classique, même s'il est très simple. En dehors du parcours d'inscription/

connexion, tous les écrans sont traités au même niveau et accessibles depuis le menu situé dans la barre latérale de navigation.

La seule exception étant les formulaires que les utilisateurs peuvent utiliser pour remplir leurs informations financières, qui sont accessible par le clic de boutons situés dans la page finances. Ces formulaires sont simplifiés au maximum pour faciliter le parcours utilisateur et limiter les erreurs humaines.

L'utilisation de donuts SVG pour l'affichage des informations est directement inspiré d'applications concurrentes, permettant à l'utilisateur de comprendre la répartition de ses revenus et dépenses et sa situation financière en un coup d'oeil.

8. CP6 — Définir l'architecture logicielle d'une application

8.1 Architecture multicouche (web / contextes / schémas)

Comme indiqué plus tôt dans ce rapport, l'architecture de LifeQuest suit la convention Phoenix de découplage des couches MVC. La couche Modèle contient les schémas Ecto et PostgreSQL, la couche Vue contient les LiveViews et les templates HEx, et la couche métier qui fait le lien entre ces éléments, grâce aux fonctions de contextes exposées dans `Finances` et `Accounts`.

Les dépendances sont unidirectionnelles : la couche web dépend des contextes, les contextes dépendent des schémas, les schémas dépendent de la base. Une LiveView ne connaît pas la structure d'une table, elle appelle une fonction du contexte qui lui retourne des données.

Cette approche apporte des avantages concrets de testabilité, de maintenance, de lisibilité, de factorisation, et permettent le travail en collaboration en évitant les conflits.

8.2 Choix du framework et de l'ORM (Phoenix / Ecto)

Ce choix s'est imposé naturellement, car c'est la stack sur laquelle je travaille en entreprise. Laissez moi tout de même vous présenter d'autres avantages concrets, au delà de la familiarité.

LiveView élimine le besoin d'un framework front et d'une API le reliant au Back, et est optimisé pour la conception d'interface réactives en temps réel, ce qui est primordial pour l'expérience utilisateur dans le cadre d'applications modernes.

Elixir s'exécute sur la BEAM, la machine virtuelle d'Erlang, qui gère la concurrence par des processus légers et isolés. Chaque utilisateur connecté dispose de son propre processus LiveView sans que sa charge n'affecte les autres.

Ecto est l'ORM de référence de l'écosystème Elixir. Il se distingue par l'absence de magie implicite : les requêtes sont composées explicitement avec `Ecto.Query`, et les données passent toujours par un changeset avant d'être écrites.

PostgreSQL, grâce à son support natif des types `decimal` pour les montants (les `float` introduisent des erreurs d'arrondi) et ses types `enum` natifs qui permettent de contraindre les valeurs directement au niveau de la base a été retenu pour sa robustesse sur les données financières,

Le serveur HTTP utilisé est Bandit, le successeur moderne de Cowboy, depuis la version 1.7 de Phoenix.

8.3 Stratégie de sécurité par couche

La sécurité est traitée à chaque couche de l'application garantissant une protection robuste et limitant les attaques possibles au maximum.

Couche Présentation : Les templates HEx échappent automatiquement toutes les variables interpolées, ce qui élimine les risques XSS par défaut. Le composant `<.form>` injecte un token CSRF dans chaque formulaire vérifié côté serveur à chaque soumission. Les routes protégées sont regroupées dans un bloc `live_session :require_authenticated_user` qui interdit l'accès sans session valide.

Couche métier : les changesets valident toutes les données avant toute écriture. Les champs système ne passent jamais par `cast/3` et sont assignés programmatiquement. Toutes les requêtes filtrent par `current_scope`, ce qui rend impossible l'accès aux données d'un autre utilisateur même en manipulant un identifiant.

Couche données : Ecto génère des requêtes paramétrées : les valeurs sont transmises séparément de la requête SQL, ce qui protège nativement contre les injections. Les enums PostgreSQL garantissent l'intégrité des valeurs sans code applicatif supplémentaire.

Côté authentication : les magic links sont des tokens à usage unique stockés sous forme hashée en base. Le token brut n'est jamais persisté, et chaque token expire après 60 jours.

8.4 Éco-conception

Cette application n'a pas particulièrement conçue avec l'écologie en tête, mais le choix de cette stack, son origine historique et les optimisations qui en découlent permettent de limiter notre consommation d'espace serveur, de puissance de calcul nécessaire et de réseau, limitant par conséquent nos besoins en électricité.

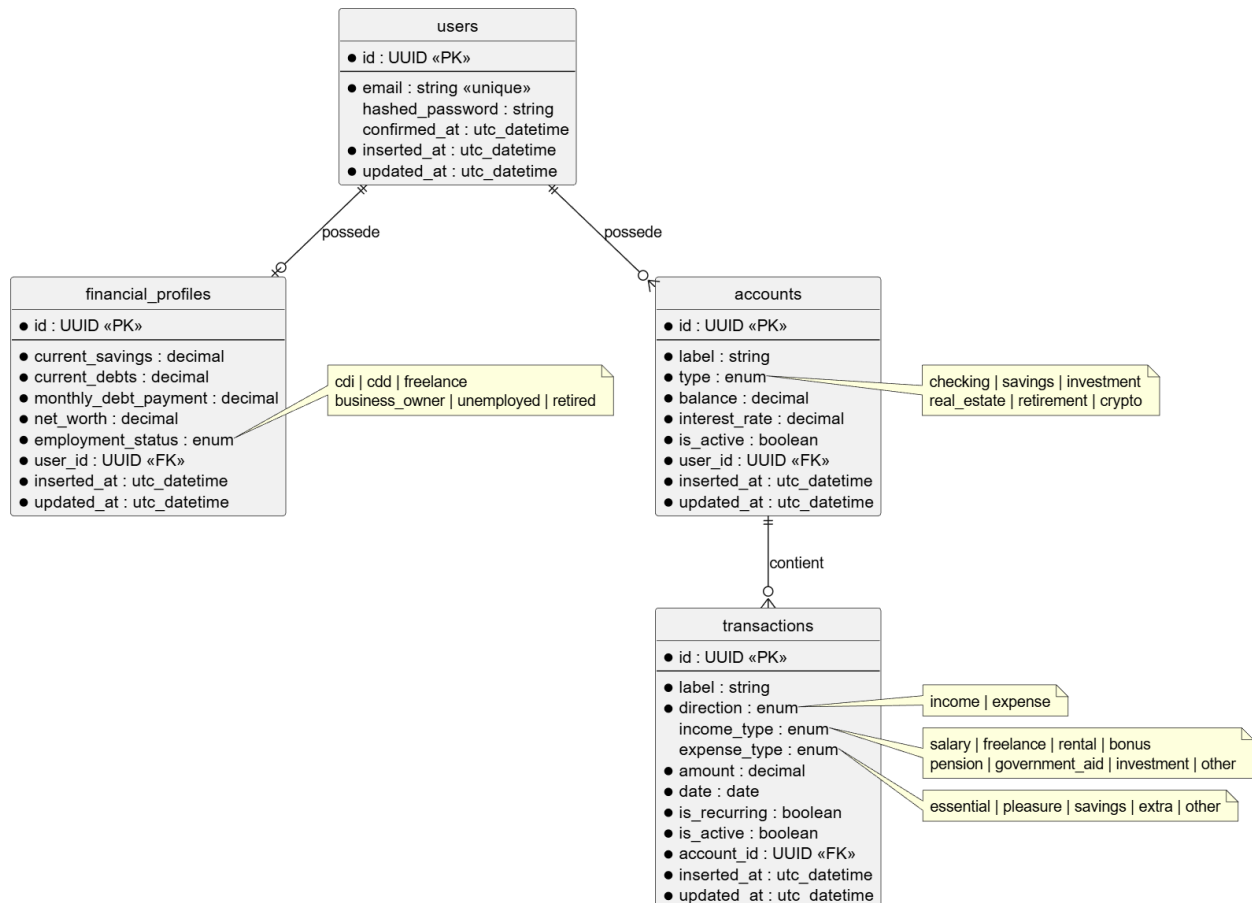
Elixir est un langage compilé. Pour la mise en production, un binaire contenant le strict nécessaire au fonctionnement de l'application est généré : code minifié, dépendances compilées, assets bundlés...

Il repose sur la BEAM, la machine virtuelle sous-jacente, qui a été conçue à l'origine pour les systèmes de télécommunications où la densité de connexions simultanées est élevée. Elle gère la concurrence par des processus légers, dont chacun n'occupe que quelques kilo-octets en mémoire au démarrage.

LiveView contribue à réduire la consommation réseau par sa conception même : le serveur calcule le diff du DOM et n'envoie via WebSocket que les parties modifiées. Chaque utilisateur maintient une seule connexion persistante, en lieu et place des multiples requêtes HTTP qu'une architecture REST classique générerait à chaque interaction.

9. CP7 — Concevoir et mettre en place une base de données relationnelle

9.1 Modèle Conceptuel de Données (MCD)



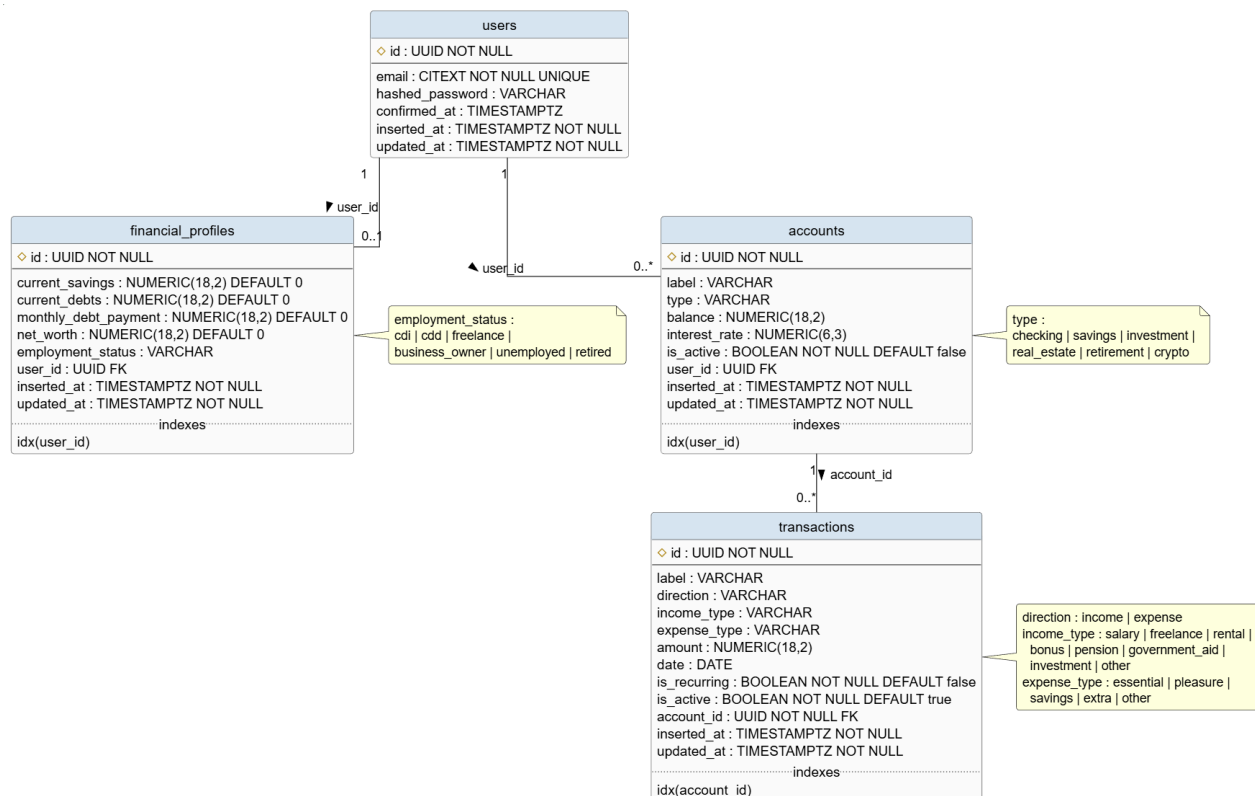
Le modèle de données de LifeQuest est relativement simple, puisqu'il s'agit d'une application qui simule ce que pourrait être une implémentation plus complexe, liée à des données externes réelles comme des comptes bancaires ou des produits financiers. Dans le cas d'utilisation d'API externe, le modèle pourrait évoluer pour s'adapter à des données réelles du monde de la finance. Dans notre cas, cette simplification est suffisante, et pourrait même peut être suffire à une implémentation concrète.

Il repose sur 4 entités, `users`, `financial_profiles`, `accounts` et `transactions`. Un utilisateur possède 0 ou 1 profil financier, 0 ou n comptes et chaque compte contient 0 à n transactions.

Dans Phoenix, ces 4 entités et leurs relations sont définies dans des schémas groupés et lisibles, ce qui permet de facilement visualiser, modifier et comprendre le modèle de données.

Au démarrage du projet, j'avais envisagé de séparer la table transactions en deux tables revenus et dépenses. J'ai rapidement constaté que cette séparation introduisait une complexité qui n'était pas nécessaire, ces deux objets partageant les mêmes attributs. J'ai choisi de les fusionner en une seule entité en ajoutant un champ `direction` (`:income` ou `:expense`) et des champs `income_type` et `expense_type` optionnels selon la direction. Cette décision sera développée dans la partie 15.1.

9.2 Modèle Logique de Données (MLD)



Ce diagramme MLD permet de visualiser la structure de données du côté de PostgreSQL, et sont représentés dans la structure de fichiers par les fichiers de migration.

Les quatre tables PostgreSQL correspondent directement aux entités du MCD. Quelques choix techniques méritent d'être soulignés.

Les clés primaires sont des UUID (`binary_id` chez Ecto) plutôt que des entiers auto-incrémentés, ce qui évite un compteur prédictible dans les URLs.

Les montants utilisent le type `NUMERIC(18, 2)` plutôt que `FLOAT`. Quand on utilise des floats, `0.1 + 0.2` ne vaut pas exactement `0.3`. Ce comportement est inacceptable pour une application financière : on utilise donc des décimaux à précision fixe.

Les champs de catégorie (`direction`, `income_type`, `expense_type`, `employment_status`, `type`) sont stockés en `VARCHAR` et validés côté application par les changesets Ecto, avec des enums définis au niveau du schéma Elixir. Les colonnes fréquemment filtrées sont indexées : `user_id` sur `financial_profiles` et `accounts`, `account_id` sur `transactions`.

9.3 Migrations Ecto et scripts de création

Ecto impose un processus strict pour les modifications de schéma : `mix ecto.gen.migration nom_migration` génère un fichier horodaté dans `priv/repo/migrations/`, que l'on édite puis applique avec `mix ecto.migrate`. Les migrations sont versionnées avec le reste du code, ce qui permet de rejouer l'historique complet du schéma sur n'importe quel environnement.

La convention est d'utiliser `change/0` qui rend la migration réversible automatiquement via `mix ecto.rollback`. On peut également utiliser `up/0` et `down/0` pour les rollbacks qui nécessitent absolument une logique personnalisée.

L'historique des migrations illustre bien l'évolution du schéma. La table `transactions` a d'abord été créée avec une clé étrangère directe vers `users`. Une migration ultérieure a supprimé ce lien pour le remplacer par une clé vers `accounts`, ce qui correspond à la décision d'architecture de passer par les comptes bancaires comme point d'entrée des transactions. Une règle stricte s'applique : on ne modifie jamais une migration déjà fusionnée sur `main`. Bien que cela ne soit pas catastrophique en environnement de dev, toute correction passe par une nouvelle migration car c'est la bonne pratique une fois que notre application tourne en production.

9.4 Sécurité et droits d'accès

L'isolation des données est garantie par la chaîne de clés étrangères : une `Transaction` appartient à un `Account`, qui appartient à un `User`. On ne récupère jamais une transaction par son seul identifiant, on filtre toujours via le scope de l'utilisateur, ce qui garantit qu'un accès par un ID manipulé retourne une erreur 404 plutôt qu'une fuite de données.

L'utilisateur utilisé par l'application dispose uniquement des droits de lecture et d'écriture sur les tables de l'application, sans droits d'administration. Il ne peut pas modifier le schéma, créer des tables ou accéder aux tables système.

9.5 Jeu d'essai

Le fichier `priv/repo/seeds.exs` permet de peupler la base de données avec des données fictives pour le dev ou des données d'initialisation pour la base de données en production. Elles utilisent `on_conflict: :nothing` pour ne pas créer de doublons si on les joue plusieurs fois. Dans le cadre de mon projet, seules des données propres aux utilisateurs sont stockées en base, donc je n'ai pas eu la nécessité d'en créer, la base de test étant peuplée par les fixtures.

10. CP8 — Développer des composants d'accès aux données SQL et NoSQL

10.1 Requêtes Ecto (CRUD sécurisé)

Comme expliqué plus tôt, l'accès aux données se fait exclusivement par le bien de requêtes Ecto, générées par le contexte. Ces dernières sont ensuite traduites en requêtes SQL paramétrées, dans un processus que je vais décrire en détail tout au long de ce chapitre.

Le contexte `Finances` expose les fonctions CRUD standard ainsi que des requêtes d'agrégations plus spécifiques. Chacune de ces fonctions utilise le Scope de l'utilisateur actif comme argument, et filtre les résultats par le `user_id` extrait de ce Scope via une jointure sur la table `accounts`.

```
def list_transactions(%Scope{} = scope) do
  Transaction
  |> join(:inner, [t], a in Account, on: t.account_id == a.id)
  |> where([t, a], a.user_id == ^scope.user.id)
  |> preload([t, a], account: a)
  |> Repo.all()
end
```

Par exemple, cette fonction permet de lister toutes les transactions propres à un utilisateur. Elle joint `transactions` et `accounts`, filtre sur `a.user_id == ^scope.user.id`, et précharge l'association `account` pour éviter le problème des N+1 requêtes.

```
def sum_all_by_category(%Scope{} = scope, direction) do
  type_field = if direction == :income, do: :income_type, else: :expense_type

  Transaction
  |> join(:inner, [t], a in Account, on: t.account_id == a.id)
  |> where([t, a], a.user_id == ^scope.user.id)
  |> where([t, a], t.direction == ^direction)
  |> group_by([t, a], field(t, ^type_field))
  |> select([t, a], {field(t, ^type_field), sum(t.amount)})
  |> Repo.all()
  |> Map.new()
end
```

Certaines requêtes d'agrégation sont plus spécifiques. Celle-ci regroupe les transactions par catégorie pour alimenter les donuts du dashboard : elle joint les tables `accounts` et `transactions` grâce à la FK présente sur cette dernière, filtre par direction (`:income` ou `:expense`), groupe dynamiquement sur le champ de type correspondant, et retourne une map `%{catégorie => total}`. Ces requêtes sont composées avec `Ecto.Query` de façon explicite, sans magie implicite.

```
## Examples
```

```
# iex> sum_all_by_category(scope, :income)
# %{salary: #Decimal<3500.00>, freelance: #Decimal<500.00>}

# iex> sum_all_by_category(scope, :expense)
# %{essential: #Decimal<1200.00>, pleasure: #Decimal<350.00>}
"""
```

10.2 Validation et contrôle des entrées côté serveur

```
def changeset(transaction, attrs, _scope) do
  transaction
  |> cast(attrs, [
    :label,
    :direction,
    :income_type,
    :expense_type,
    :amount,
    :date,
    :is_recurring,
    :is_active,
    :account_id
  ])
  |> validate_required([:account_id, :label, :direction, :amount, :date])
  |> validate_type_by_direction()
end
```

Toute écriture en base passe par un `changeset`, qui est chargé d'hydrater un objet complet ou partiel, ici de `transaction`, et d'exécuter un certain nombre de vérifications de base ou personnalisées. Le `changeset`

`Transaction.changeset/2` commence par `cast/3` pour lister les champs autorisés, puis enchaîne les validations : `validate_required` pour les champs obligatoires, et une fonction custom privée `validate_type_by_direction/1`

qui applique des règles conditionnelles selon la direction de la transaction. Si `direction` est `:income`, le champ `income_type` est requis et `expense_type` est forcé à `nil`. L'inverse s'applique pour `:expense`.

```
defp validate_type_by_direction(changeset) do
  case get_field(changeset, :direction) do
    :income ->
      changeset
      |> validate_required([:income_type])
      |> put_change(:expense_type, nil)

    :expense ->
      changeset
      |> validate_required([:expense_type])
      |> put_change(:income_type, nil)

    _ ->
      changeset
  end
end
```

Ces validations sont faites du côté serveur, et sont par conséquent impossible à contourner côté client. Si le changeset est invalide, `Repo.insert/2` ou `Repo.update/2` retourne `{:error, changeset}`. Le changeset de retour contient le changeset d'origine et les messages d'erreur par champ qui ont été ajoutés côté serveur. La fonction `to_form/1` fournie par Phoenix utilise ce changeset pour mettre à jour le formulaire à chaud grâce à LiveView, et afficher les erreurs tout en conservant les champs remplis par l'utilisateur, sans recharger la page.

10.3 Gestion de l'intégrité (transactions Ecto)

Certaines opérations impliquent plusieurs écritures en base qui doivent réussir ou échouer ensemble. Sans précaution, un crash entre deux opérations laisserait la base dans un état incohérent, comme des données orphelines par exemple.

Pour s'assurer d'obtenir un état final stable, Ecto fournit `Repo.transaction/1` :
Si l'une des opérations contenues dans la transaction lève une exception ou retourne une erreur, PostgreSQL annule l'ensemble de la transaction.

```
def delete_all_user_data(%Scope{} = scope) do
  user_id = scope.user.id

  Repo.transaction(fn ->
    Repo.delete_all(from(a in Account, where: a.user_id == ^user_id))
    Repo.delete_all(from(fp in FinancialProfile, where: fp.user_id ==
  end)

  :ok
end
```

Dans ce cas de figure, il s'agit de la fonction utilisée lorsqu'un utilisateur souhaite supprimer l'intégralité de ses données. Bien qu'ici on ne se retrouverait pas avec des données orphelines car les deux tables portent des clés étrangères liées à User et qu'il n'est pas question de supprimer l'utilisateur, il s'agit tout de même d'une fonctionnalité critique, et il est judicieux de s'assurer que tout est bien supprimé avant de retourner une confirmation à l'utilisateur. Cette structure permet de garantir que le feedback visuel rendu à l'utilisateur après sa manipulation sera cohérent avec l'état de la base de données.

10.4 Tests des accès aux données

Les tests du contexte `Finances` couvrent quatre axes pour chaque fonction publique : le comportement nominal, la gestion des erreurs, l'isolation par scope, et les cas limites.

Comportement nominal : Chaque fonction CRUD est testée avec des données valides et ses valeurs de retour sont vérifiées champ par champ. Le test de `create_transaction/2` vérifie par exemple que `expense_type` est bien `nil` quand on crée un revenu :

Gestion des erreurs : On utilise des attributs invalides pour vérifier que la fonction retourne bien un tuple `{:error, %Ecto.Changeset{}}`. Après l'échec, on récupère l'enregistrement en base et on vérifie qu'il est identique à l'original ce qui assure qu'on a pas fait d'écriture partielle.

Isolation par scope : Pour les lectures, on appelle la fonction avec deux scopes distincts et on vérifie que chaque scope ne voit que ses propres enregistrements. Pour les opérations ciblées on vérifie que tenter d'agir sur la ressource d'un autre utilisateur lève une exception.

Cas limites : Dans certains cas, l'entrée est valide mais sa valeur implique un changement de comportement de la logique métier. Il est primordial d'essayer de les visualiser, traiter et tester en amont, pour essayer d'éviter de les découvrir en production.

Par exemple, `project_savings/2` permet de créer une projection de l'épargne de l'utilisateur dans le futur. Ses paramètres d'entrée sont l'épargne de départ, la capacité d'épargne (revenus mensuels - dépenses mensuelles) et les mensualités de remboursement des dettes. Lorsque le montant restant de la dette est inférieur à la mensualité prévue, on entre dans un cas limite : sans précaution, on déduirait la mensualité complète sur le mois suivant, entraînant une baisse de l'épargne projetée. Le test permet de vérifier que le remboursement est plafonné au montant de dette restante, et que la mensualité de la dette passe à 0 au mois suivant.

```

test "caps debt payment at remaining debt when debt is nearly paid" do
  scope = user_scope_fixture()

  financial_profile_fixture(scope, %{
    current_savings: "1000.00",
    current_debts: "150.00",
    monthly_debt_payment: "100.00"
  })

  {:ok, projection} = Finances.project_savings(scope, months_ahead(3))
  # Month 1: savings=900, debts=50    (payment=min(100,150)=100)
  # Month 2: savings=850, debts=0    (payment=min(100,50)=50)
  # Month 3: savings=850, debts=0    (payment=min(100,0)=0)
  assert Decimal.equal?(projection.projected_savings, Decimal.new("850.00"))
  assert Decimal.equal?(projection.projected_debts, Decimal.new("0"))
end

```

10.5 Protection contre l'injection SQL

L'injection SQL est l'une des vulnérabilités les plus répandues dans les applications web. Elle consiste à insérer du SQL malveillant dans une valeur d'entrée pour manipuler la requête construite côté serveur. La parade classique est d'utiliser des requêtes paramétrées : la valeur est transmise séparément de la requête, PostgreSQL ne l'interprète jamais comme du SQL.

Ecto applique ce principe nativement, sans configuration. L'opérateur `^` marque une valeur comme paramètre externe :

```
|> where([t, a], a.user_id == ^scope.user.id)
```

Ce code génère le SQL suivant :

```
WHERE accounts.user_id = $1
```

La valeur de `scope.user.id` est transmise à PostgreSQL dans un second canal, distinct de la requête elle-même. Peu importe ce que contient cette valeur, elle sera toujours traitée comme une donnée et jamais interprétée. Sans le `^`, Elixir tenterait d'interpoler la valeur directement dans la requête à la compilation, ce qui serait refusé par Ecto avec une erreur explicite.

Cette protection est donc structurelle : elle ne dépend pas de la vigilance du développeur sur chaque requête, mais du comportement par défaut de l'ORM. L'injection SQL via les fonctions Ecto standard est impossible dans LifeQuest.

11. CP9 — Préparer et exécuter les plans de tests d'une application

11.1 Plan de tests

Grâce à la méthode de travail acquise chez Frixel, la rédaction d'un plan de tests n'a pas été nécessaire durant le développement de mon application. En effet, le framework de tests inclus dans Phoenix et la dépendance coveralls permettant de monitorer la couverture de tests et exécutée dans la CI nous pousse à garder une couverture de tests quasi complète sur toute la durée du développement. Le comportement que cela induit est simple : chaque branche doit apporter les tests des fonctionnalités et LiveViews qu'elle ajoute au projet.

La nature des tests elle aussi est induite par le framework, et a été acquise au cours de mon expérience en entreprise. Les contextes font l'objet de tests unitaires pour chacune de leurs fonctions, et chaque LiveView fait l'objet de tests d'intégration. Les critères de réussite sont simples : aucun test ne doit échouer, un warning est considéré comme une erreur, et la CI doit passer avant tout merge sur main.

L'ensemble des tests sont couverts sur 2 points au minimum, le comportement du cas nominal et l'isolation des données par scope. Certaines fonctionnalités ayant des implémentations plus spécifiques peuvent nécessiter d'être testées sur des cas limites, lorsqu'on parvient à les détecter.

Vous trouverez en Annexe 4.1 un tableau non-exhaustif des tests de l'application, pour vous donner un aperçu compréhensible des fonctionnalités testées.

11.2 Tests unitaires (ExUnit)

Elixir intègre son propre framework de tests unitaires `ExUnit` que Phoenix permet d'utiliser de manière poussée grâce à des modules complémentaires. ExUnit nous permet d'écrire et de jouer des tests dans un bac à sable qui les isole les uns des autres, permettant d'effectuer chacun des tests dans une base de test, avec un état de départ déterminé.

Chaque test s'exécute dans une transaction qui rollback automatiquement une fois le test terminé ce qui garantit qu'un test ne pourra pas influencer sur le résultat d'un autre test, même lorsqu'ils sont joués en parallèle.

La structure des tests dans Lifequest est standard : Un bloc `describe` permet de regrouper les tests, et chaque test porte pour nom une phrase en anglais qui décrit clairement le scope du test, sous la forme "la fonction X dans la situation Y a le comportement Z"

Par exemple, *`update_transaction/3 with invalid scope raises`*.

Pour éviter de répéter la création des données de test dans chaque fonction, des fixtures réutilisables sont définies dans les fichiers qui y sont consacrés. La structure de base d'une fixture crée un objet en base avec des valeurs par défaut, mais l'ajout d'une map en argument facultatif permet d'écraser les valeurs de notre choix, en fonction des besoins du test.

Exemples concrets : *Voir annexe 3.2 et 3.3*

L'implémentation des tests d'isolation par scope peut être implémentée de deux façons, en fonction des valeurs de retour des fonctions testées. Ces deux fonctions créent des conditions de base similaires, avec la création de deux utilisateurs en base, et de transactions qui leur sont associées. `list_transactions/1` retourne une liste de transactions qui peut être vide, donc on vérifie que lorsqu'un utilisateur l'exécute il obtient bien et seulement les transactions qui lui sont associées. `get_transaction!/2` retourne une transaction ou une erreur. Cela est indiqué par le `bang operator !`, une convention pour nommer les fonctions qui ont ce genre de comportement. On utilise donc `assert_raise Ecto.NoResultsError` pour vérifier que tenter d'accéder à la transaction d'un autre utilisateur lève une exception.

11.3 Tests d'intégration (LiveView)

Les LiveViews sont la base de la couche présentation. Toutes leurs fonctionnalités sont testées grâce à des tests d'intégration dans lesquels la connexion HTTP est simulée grâce au module `ConnCase`.

Il y a 3 types de tests différents pour les LiveViews :

- **Test du rendu initial :** `{:ok, _live, html} = live(conn, ~p"/dashboard")` monte la LiveView et retourne le HTML rendu. On vient faire des assertions textuelles, pour vérifier que le contenu est bien chargé dans le DOM.
- **Tests de clics :** `live |> element("button", "Valider") |> render_click()` simule un clic sur un élément du DOM et retourne le nouveau rendu après l'événement.
- **Tests de formulaires :** `live |> form("form", transaction: %{...}) |> render_submit()` simule une soumission complète et permet de vérifier le résultat.

Exemples concrets : Voir annexe 2.2

11.4 Tests de sécurité

Deux tests majeurs sont effectués pour s'assurer du bon fonctionnement des couches de sécurité de base du projet.

- Les tests d'isolation par scope pour les données confidentielles, que j'ai déjà eu l'occasion de vous présenter à plusieurs reprises tout au long de ce rapport. Pour chaque LiveView qui affiche des données, un test crée des données pour un second utilisateur (`other_scope`) et vérifie qu'elles n'apparaissent pas dans le rendu de l'utilisateur courant grâce à `refute html =~ "label de l'autre utilisateur"`. Ces tests valident simultanément le comportement fonctionnel et la sécurité des données.
- Les tests de redirection pour les LiveViews accessibles aux utilisateurs identifiés seulement. Ce test est présent dans le fichier de test de chaque LiveView pour lequel c'est nécessaire. Il crée une connexion sans session

grâce à `build_conn()`, et vérifie que l'utilisateur est redirigé vers la page de connexion. Ce test est systématique : il protège contre les régressions où une route serait accidentellement déplacée hors du `live_session :require_authenticated_user`.

```
assert {:error, {:redirect, %{to: path}}} = live(build_conn(), ~p"/dashb
assert path == ~p"/users/log-in"
```

11.5 Résultats et compte-rendu de tests

Le projet compte 262 tests répartis sur 21 fichiers, avec 0 failure et 0 warning de compilation. La couverture porte sur l'ensemble des fonctions publiques des contextes `Finances` et `Accounts`, toutes les LiveViews de l'application (dashboard, finances, comptes, projections, placements, objectifs, paramètres, authentification), et les tests de sécurité associés.

Les fonctions privées (`defp`), les callbacks internes de LiveView et le comportement des librairies tierces (Ecto, Phoenix) sont volontairement exclus du périmètre de test.

La CI GitHub Actions exécute la suite complète à chaque push et à chaque PR vers `main`, contre une vraie base PostgreSQL 16. Aucun merge n'est autorisé si la CI est rouge.

12. CP10 — Préparer et documenter le déploiement d'une application

12.1 Environnements (développement, test, production)

Phoenix permet de gérer nativement différents environnements en fonction du contexte d'exécution. Il en existe 3 par défaut `dev`, `test` et `prod`, qui sont définis dans le dossier `config/` et chargés grâce à une variable d'environnement `MIX_ENV`.

L'environnement de **développement** (`config/dev.exs`) est configuré pour maximiser le confort de travail : il utilise la base de données locale, permet de voir

les changements de code sans redémarrer le serveur grâce au hot reload, et le mailer permet de simuler les envois de mails sur une boîte mail fictive. Les logs sont détaillés pour faciliter le débogage.

L'environnement de **test** (`config/test.exs`) est conçu pour l'isolation et la reproductibilité. Il utilise une base de données séparée de celle du développement, ce qui garantit que les tests n'interagissent pas avec les données de la base de dev ou, cela va sans dire, de prod. Chaque test se exécute dans une transaction rollbackée automatiquement et le mailer est configuré en mode mémoire, sans envoi réel.

L'environnement de **production** est géré par deux fichiers complémentaires.

`config/prod.exs` est chargé à la **compilation** pour fixer les valeurs statiques connues à ce moment là, puis `config/runtime.exs` est chargé au **démarrage** de l'application. Ce fichier permet de lire les variables d'environnement grâce à `System.get_env`, qui seront accessibles dans le reste de l'application via `Application.get_env`. La configuration est ainsi centralisée, testable et ne fuite pas dans les logs.

12.2 Procédure de déploiement

Pour déployer l'application, j'ai décidé d'utiliser Fly.io qui est la plateforme de référence pour l'écosystème Phoenix. Après l'avoir installé, la commande `fly launch` a analysé le projet et généré automatiquement le `Dockerfile` et le fichier de configuration `fly.toml`.

La procédure de déploiement manuel est la suivante :

- S'authentifier avec `flyctl auth login`
- Depuis la racine du projet, lancer `flyctl deploy` qui construit l'image Docker sur les serveurs de Fly.io et la déploie
- Fly.io exécute automatiquement les migrations de base de données avant de démarrer la nouvelle version

En pratique, cette commande n'est jamais lancée manuellement en dehors des phases initiales : c'est la pipeline CD qui s'en charge automatiquement à chaque merge sur `main`, comme décrit dans la partie 13.

12.3 Scripts de déploiement et migrations

Pour présenter cette partie, laissez moi vous expliquer le concept de release OTP. Elixir est un langage compilé qui s'exécute sur la BEAM. `mix release` permet de générer un binaire autonome contenant le runtime Erlang, l'application compilée, ses dépendances et les assets, sans nécessiter qu'Elixir ou Mix soient installés sur la machine cible. C'est ce binaire qui est embarqué dans le stage final du Dockerfile et déployé sur Fly.io.

Le Dockerfile suit un pattern multi-stage permettant de produire des images de production légères. Un premier stage, `builder`, installe les outils de compilation, récupère les dépendances, compile les assets et génère la release. Un second stage, `final`, repart d'une image Debian minimale et ne copie que la release produite par le premier stage. L'image finale ne contient donc aucun outil de build, ce qui réduit sa taille et sa surface d'attaque.

Le répertoire `rel/` contient les scripts de démarrage générés par `mix release`. Le script `server` démarre l'application, et le script `migrate` exécute les migrations Ecto en base de données. Ce dernier est configuré comme `release_command` dans le `fly.toml` :

```
[deploy]
  release_command = '/app/bin/migrate'
```

Fly.io exécute ce script avant chaque démarrage d'une nouvelle version de l'application. Les migrations sont donc toujours jouées avant que le trafic ne soit basculé vers la nouvelle version, ce qui garantit la cohérence entre le schéma de base de données et le code déployé.

12.4 Variables d'environnement et configuration

LifeQuest requiert quatre variables d'environnement en production :

- `SECRET_KEY_BASE` : clé secrète utilisée pour signer les sessions et les tokens.
- `DATABASE_URL` : URL de connexion à la base de données PostgreSQL.
- `PHX_HOST` : domaine public de l'application.

- `PORT` : port sur lequel l'application écoute les connexions entrantes.

Ces variables sont configurées côté Fly.io via la commande `flyctl secrets set NOM=valeur`. Elles sont stockées de manière chiffrée sur la plateforme et injectées dans l'environnement du conteneur au démarrage, sans jamais apparaître dans le code ou les logs.

Le fichier `.env.example` documente ces variables avec leurs descriptions, sans aucune valeur réelle. Ce fichier est versionné dans le dépôt Git, ce qui permet à tout développeur reprenant le projet de savoir exactement quelles variables configurer pour lancer l'application.

13. CP11 — Contribuer à la mise en production dans une démarche DevOps

13.1 Pipeline CI/CD (GitHub Actions)

La pipeline est définie dans `.github/workflows/ci-cd.yml` et se déclenche à chaque push et à chaque PR ouverte vers `main`. Elle est composée de deux jobs distincts : `ci` et `deploy`.

Le job `ci` s'exécute en premier, et dans un ordre précis : vérification du formatage, analyse statique Credo, compilation avec `--warnings-as-errors`, puis exécution des tests. Les vérifications rapides sont faites en premier pour échouer rapidement et éviter d'attendre l'exécution complète des tests si le code n'est pas formaté correctement. Les tests arrivent en dernier car ils sont les plus longs à s'exécuter.

Pour que les tests puissent s'exécuter dans la CI exactement comme en local, le runner GitHub Actions démarre une instance PostgreSQL 16 sous forme de service Docker, avec un healthcheck qui garantit que la base est prête avant de lancer les tests.

```
services:
  postgres:
    image: postgres:16
    options: >-
      --health-cmd pg_isready
      --health-interval 10s
```

Le job `deploy` ne s'exécute que sur un push vers `main`, jamais sur une PR. Il est conditionné à la réussite du job `ci` grâce à `needs: [ci]`, ce qui garantit que seul du code validé est déployé en production.

13.2 Conteneurisation (Docker)

Comme décrit dans la partie 12.3, `fly launch` a généré un Dockerfile multi-stage, ce qui est intéressant d'un point de vue DevOps : l'image finale déployée en production ne contient aucun outil de compilation, aucune version d'Elixir ou de Mix. Elle ne contient que le binaire de la release OTP et les librairies système dont la BEAM a besoin pour s'exécuter, ce qui se traduit par une image plus légère et une surface d'attaque réduite.

Le projet dispose également d'un `docker-compose.yml` qui permet de lancer l'application complète en local sans aucune installation d'Elixir ou de PostgreSQL. Il orchestre deux services : `db` (PostgreSQL 16 avec un volume persistant et un healthcheck) et `app` (construit à partir du Dockerfile local). Le service `app` dépend de la bonne santé du service `db` avant de démarrer, ce qui évite les erreurs de connexion au démarrage.

```
depends_on:
  db:
    condition: service_healthy
command: /bin/sh -c "/app/bin/migrate && /app/bin/seed && /app/bin/serve"
```

La commande enchaîne trois étapes : migrations, seeds, puis démarrage du serveur. Les seeds sont idempotents grâce à `on_conflict: :nothing`, ce qui permet de les rejouer sans risque à chaque démarrage. Ils créent un compte de

démonstration (`demo@lifequest.fr` / `LifeQuest2025!`) pour permettre de tester l'application sans configuration supplémentaire. Une seule commande suffit pour lancer l'environnement complet :

```
docker compose up --build
```

13.3 Outils de qualité de code (Credo, mix format)

Trois outils s'enchaînent dans la CI pour garantir la qualité du code, et je les applique également en local avant chaque commit grâce à un alias `mix precommit` défini dans le projet.

`mix format` est le formateur officiel de l'écosystème Elixir. Il n'a aucune configuration à maintenir : son comportement est déterministe et partagé par tous les projets Elixir. La vérification `mix format --check-formatted` échoue si le code n'a pas été formaté, ce qui élimine tout débat de style dans les revues de code.

`mix credo --strict` est un analyseur statique qui détecte les problèmes de style et les anti-patterns Elixir : fonctions trop complexes ou trop longues, modules sans documentation, variables inutilisées... Le mode `--strict` active toutes les vérifications disponibles. J'ai utilisé cet outil tout au long du développement pour m'assurer d'avoir un code propre à chaque étape.

`mix compile --warnings-as-errors` transforme les avertissements de compilation en erreurs bloquantes. En Elixir, un warning peut signaler une clause de pattern matching inaccessible ou une variable non utilisée, deux situations qui révèlent souvent un bug latent. En les traitant comme des erreurs dès la CI, on garantit qu'ils ne s'accumulent pas.

13.4 Automatisation des tests

les tests sont exécutés automatiquement à chaque push par GitHub Actions, sans aucune intervention manuelle. Ce qui est spécifique à la CI, c'est l'utilisation d'une vraie base PostgreSQL 16, identique à celle de production, plutôt qu'un mock ou une base en mémoire. Cette approche garantit que les requêtes Ecto fonctionnent exactement comme en production.

13.5 Interprétation des rapports CI

Chaque job est visible dans l'interface GitHub, avec le détail de chaque étape. Lorsqu'une étape échoue, son log s'affiche directement : il contient le fichier, la ligne concernée, et le message d'erreur exact, ce qui permet d'identifier et de résoudre facilement les problèmes.

14. Communiquer en français et en anglais

14.1 Documentation technique en anglais

Dans le développement de LifeQuest, l'anglais est la langue du code et la langue de la documentation technique. Tous les identifiants du projet suivent cette règle : les noms de fonctions (`list_transactions` , `get_transaction!` , `deliver_login_instructions`), les modules (`FinancialProfile` , `UserToken` , `Scope`), les variables (`current_scope` , `socket` , `changeset`) et les noms de routes sont en anglais. C'est une convention universelle dans l'écosystème Elixir et dans le développement logiciel en général que j'ai appliquée dès le premier commit.

Les fichiers de conventions et de configuration du projet sont également rédigés en anglais, ce qui permet à n'importe quel développeur international de comprendre les règles du projet sans barrière linguistique.

Les messages de commit suivent une convention hybride : le type et le scope sont en anglais (`feat` , `fix` , `dashboard` , `transactions`), et la description est en français. Par exemple : `LQ-50 feat(dashboard): donuts SVG mensuels` . Ce choix permet à la fois de respecter les conventions internationales de commit et de maintenir une lisibilité en français pour le suivi de projet.

14.2 I18n avec Gettext

L'interface de LifeQuest est rédigée en anglais dans le code source, et traduite en français via le module `Gettext` intégré à Phoenix. C'est une pratique standard dans l'écosystème : le code source reste en anglais, langue universelle du développement, et les traductions sont externalisées dans des fichiers dédiés.

Le fonctionnement est simple : dans les templates et les LiveViews, chaque chaîne visible par l'utilisateur est passée à la fonction `gettext/1` plutôt qu'écrite en dur.

```
{gettext("Projected savings")}  
{gettext("Projection over %{months} months", months: @projection.months)}
```

La commande `mix gettext.extract --merge` parcourt automatiquement le code source, extrait toutes les chaînes marquées avec `gettext`, et génère les fichiers `.pot` (templates de traduction) et `.po` (traductions par langue) dans `priv/gettext/`. Le fichier `priv/gettext/fr/LC_MESSAGES/default.po` contient l'ensemble des traductions françaises du projet.

```
msgid "Projected savings"  
msgstr "Épargne projetée"  
  
msgid "%{years} years"  
msgstr "%{years} ans"
```

La locale par défaut est configurée dans `config/config.exs` :

```
config :lifequest, LifequestWeb.Gettext, default_locale: "fr"
```

Ce mécanisme garantit que l'application est présentée en français aux utilisateurs, tout en conservant un code source entièrement en anglais. Il prépare aussi l'application à une éventuelle internationalisation : ajouter une nouvelle langue ne nécessite que d'ajouter un fichier `.po` correspondant, qui pourra facilement être donné à un traducteur pour lui permettre de travailler.

14.3 Lecture de documentation technique en anglais

La totalité de la documentation technique que j'ai consultée pour construire LifeQuest est en anglais. Les sources principales sont `hexdocs.pm` pour Phoenix, Ecto, LiveView et leurs dépendances, ainsi que les guides officiels Phoenix et la documentation de Tailwind CSS.

Plus généralement, les forums techniques comme ElixirForum sont des ressources que j'utilise régulièrement pour débloquer des problèmes. Les discussions y sont quasi exclusivement en anglais, et la capacité à formuler une recherche précise, à lire les réponses et à identifier la solution pertinente est une compétence que j'ai développée au quotidien pendant ce projet.

15. Mettre en œuvre une démarche de résolution de problème

La planification et la conception sont autant d'outils pouvant servir à limiter les problèmes de dev. Malgré cela, il arrive parfois qu'on ait à faire face à des imprévus. Quand cela arrive, le processus itératif permet de limiter l'impact de ces surprises, et un bon développeur se reconnaît moins à sa capacité à éviter les problèmes qu'à sa méthode pour les résoudre : identifier le symptôme, formuler des hypothèses, les tester, et tirer une leçon réutilisable. Je vais vous présenter trois situations concrètes rencontrées pendant le développement de LifeQuest, avec à chaque fois le contexte, l'analyse et la solution retenue.

15.1 Refactoring du modèle de données (LQ-5)

Au démarrage du projet, les revenus et les dépenses étaient modélisés dans deux entités distinctes : `IncomeStream` et `Expense`. Cette séparation semblait logique au premier abord, et mon approche instinctive du modèle de données sans passer par un dictionnaire de données m'a poussé à faire ce choix de conception.

Lorsque j'ai commencé à implémenter mes fonctions de prédiction, les requêtes Ecto devenaient complexes, difficiles à tester et à maintenir.

Après analyse, notamment une comparaison des deux schémas de données, j'ai décidé de fusionner les deux entités en une table `Transaction` unifiée avec un champ `direction` (`:income` / `:expense`). Cette migration a simplifié l'ensemble du contexte `Finances`, réduit le contexte de moitié et rendu les requêtes d'agrégation directes.

15.2 Encodage des apostrophes dans les tests LiveView (LQ-65)

Lors de la traduction de l'interface en français grâce à gettext (LQ-65), les tests ont de LiveViews qui étaient rédigés en anglais ont naturellement échoué. Après traduction de ces tests, les assertions du type `assert html =~ "S'inscrire"` n'étaient jamais vérifiées malgré la bonne présence du texte dans le navigateur.

L'investigation a révélé que `Phoenix.LiveViewTest` retourne du HTML brut dans lequel les apostrophes sont encodées en entité HTML (`'`). L'assertion attendait le caractère littéral, absent du rendu. Cela a été facile à diagnostiquer, car les tests invalides retourne une comparaison directe entre les chaînes de caractères.

La solution : remplacer toutes les assertions contenant des apostrophes par leur équivalent encodé (`"S'inscrire"`). La leçon : dans les tests LiveView, le rendu est du HTML échappé, pas du texte brut et il faut adapter les assertions en conséquence.

15.3 Diagnostic d'une CI rouge inattendue

Après un merge en apparence anodin, la CI GitHub Actions s'est mise à échouer sur l'étape `mix compile --warnings-as-errors`. En local, la compilation passait sans avertissement.

L'analyse du rapport CI a révélé un warning sur une variable non utilisée, dans un fichier qui n'avait pas été recompilé à cause du cache. Forcer `mix compile --force` en local a reproduit l'erreur.

Cette expérience sans conséquence a tout de même servi à démontrer de l'importance de l'accumulation des garde-fous, même s'ils peuvent sembler parfois rébarbatifs. Les différences d'environnement peuvent entraîner des problèmes qu'on ne détecte pas en local.